

DROSOPHILA-LANG: UN LENGUAJE DE PROGRAMACIÓN ESOTÉRICO QUE SE EJECUTA EN EL CEREBRO DE UNA MOSCA

AKETZA BARAJAS GARRIDO, AQUILES RIVERO LÓPEZ
Y FLORENTIN VIOREL DELGADO NAVARRO

RESUMEN. DROSOPHILA-LANG es un lenguaje de programación esotérico cuyos programas se ejecutan sobre el mapa completo del cerebro de la mosca de la fruta que publicó el proyecto FlyWire: 138 639 neuronas y 2 700 513 conexiones reales. Cada paso del programa se asigna a una neurona real, cada variable es la fuerza de una conexión que la dopamina refuerza o debilita, y el programa termina cuando dispara la neurona con la que la mosca huye. Este documento define el lenguaje y el modelo de neurona con el que se ejecuta, demuestra que es Turing-completo por reducción a las máquinas de Minsky, acota la probabilidad de que el ruido neuronal cambie el resultado y discute qué parte del cálculo hace de verdad el cerebro de la mosca.

1. INTRODUCCIÓN

Un lenguaje de programación es una forma de escribir órdenes que una máquina sigue al pie de la letra. Un lenguaje *esotérico* es uno inventado para explorar una idea, no para usarlo en el trabajo: suele tener muy pocas instrucciones o una forma extraña de ejecutarse, y precisamente por eso obliga a preguntarse qué hace falta, como mínimo, para que algo pueda calcular.

La idea de DROSOPHILA-LANG es que quien ejecute las órdenes sea el cerebro de una mosca. En 2024, el proyecto FlyWire publicó el primer mapa completo del cerebro de un animal adulto: todas las neuronas de la mosca de la fruta y todas las conexiones entre ellas [1], junto con el tipo de cada neurona [2]. Ese mismo año, Shiu *et al.* mostraron que, simulando ese mapa en un ordenador, se reproducen respuestas reales de la mosca [3]. DROSOPHILA-LANG usa esos datos: cada pieza de un programa se asigna a una neurona real con un papel parecido, y ejecutar el programa es simular el cerebro paso a paso.

La idea de que las neuronas pueden calcular es antigua: en 1943, McCulloch y Pitts ya describieron la neurona como una puerta lógica [4]. Lo nuevo aquí es hacerlo sobre un cerebro concreto, con sus tipos de neurona y sus limitaciones, y comprobar hasta dónde llega.

El documento sigue el recorrido de un programa. El Apartado 2 explica la parte del cerebro que usa el lenguaje; el Apartado 3, cómo se escribe un programa; el Apartado 4, el modelo de neurona con el que se ejecuta; el Apartado 5 demuestra que el lenguaje puede calcular cualquier cosa; el Apartado 6 muestra qué pasa al ejecutarlo de verdad; y el Apartado 7 discute qué parte del cálculo hace realmente la mosca.

2. LA MOSCA EN CINCO MINUTOS

Para entender el lenguaje basta con conocer un recorrido del cerebro de la mosca: el que va desde que la antena detecta un olor hasta que la mosca responde. La Figura 1 lo resume, junto con el papel que cada neurona desempeña en DROSOPHILA-LANG.

Fecha: 2 de octubre de 2026. Código: github.com/stavara/drosophila-lang, commit c0a58bf.

- (1) **Receptores olfativos (ORN).** Las neuronas de la antena detectan moléculas del aire. Todas las que detectan el mismo tipo de olor envían sus axones al mismo punto del lóbulo antenal, una pequeña esfera llamada *glomérulo*.¹
- (2) **Neuronas de proyección (PN).** Desde cada glomérulo, unas pocas neuronas de proyección llevan la señal hacia dos centros superiores: el cuerpo en champiñón y el cuerno lateral.
- (3) **Cuerpo en champiñón: células de Kenyon (KC) y MBON.** Es el centro de aprendizaje olfativo. Unas cinco mil células de Kenyon reciben la señal de las neuronas de proyección y la transmiten a unas pocas neuronas de salida, las MBON (del inglés *mushroom body output neurons*). La fuerza de cada sinapsis $KC \rightarrow MBON$ puede cambiar: así se guarda lo que la mosca aprende de un olor [6].
- (4) **Neuronas dopaminérgicas (PAM y PPL1).** Liberan dopamina sobre las sinapsis $KC \rightarrow MBON$ y modifican su fuerza. Hay dos familias que actúan en compartimentos distintos, llamadas PAM y PPL1 por la posición de sus cuerpos celulares (*protocerebral anterior medial* y *protocerebral posterior lateral*) [6].²
- (5) **Cuerno lateral (LH).** Recibe también la señal olfativa y contiene neuronas locales que la combinan. DROSOPHILA-LANG las usa como compuertas lógicas.
- (6) **Salidas motoras.** La motoneurona MN9 extiende la probóscide, la «trompa» con la que la mosca come [8]; y la Fibra Gigante (DNp01; las siglas DN vienen de *descending neuron*) desencadena la huida, el salto con el que la mosca escapa de un peligro [9]. DROSOPHILA-LANG usa además otras motoneuronas del cerebro y la neurona descendente DNa02 como salida de datos.

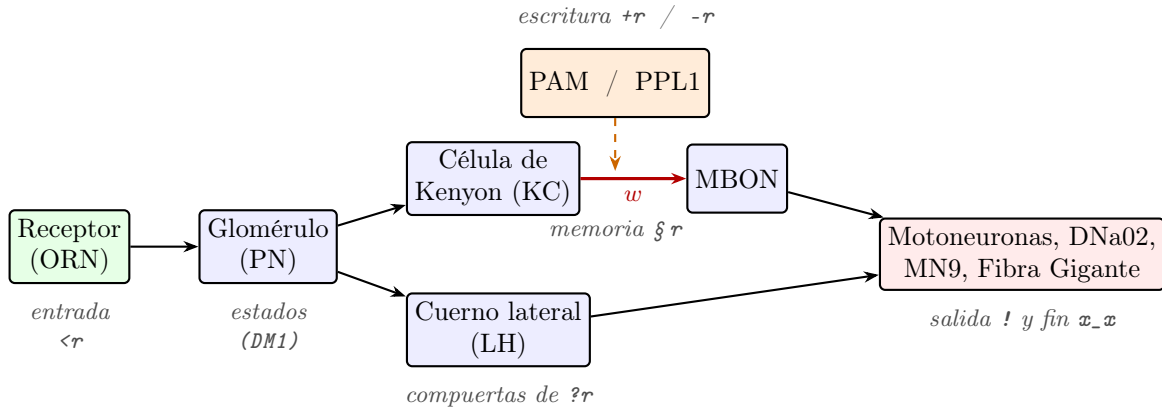


FIGURA 1. Recorrido del olor en el cerebro de la mosca (cajas) y papel que desempeña cada tipo de neurona en DROSOPHILA-LANG (cursiva). La flecha roja es la sinapsis $KC \rightarrow MBON$, cuya fuerza w guarda un número; la dopamina (línea discontinua) la modifica.

¹La mosca tiene unos cincuenta glomérulos con nombres como DA1, DM1 o VA2, que indican su posición (dorsal, anterior, ventral...). El glomérulo DA1, por ejemplo, responde a la feromona cVA [5].

²En la mosca real, la dopamina *debilita* la sinapsis $KC \rightarrow MBON$ de las células de Kenyon que respondían al olor en ese momento [7]. DROSOPHILA-LANG simplifica: las PAM suman uno y las PPL1 restan uno, para tener un contador que sube y baja.

Todas estas neuronas existen en el conectoma de FlyWire con su tipo celular anotado: hay 5177 células de Kenyon, 514 neuronas locales del cuerno lateral, 307 PAM, 96 MBON y solo 16 PPL1. Estas cifras limitarán el tamaño de los programas (Apartado 5.3).

3. EL LENGUAJE

Definición 3.1 (Programa). *Un programa de DROSOPHILA-LANG es una tupla $P = (Q, R, q_0, \delta)$, donde Q es un conjunto finito de estados (glomérulos), R un conjunto finito de variables (engramas) con valores en $\mathbb{N}$, $q_0 \in Q$ el estado inicial (el soplo) y δ asigna a cada estado una secuencia finita de acciones seguida de un desenlace: un salto a otro estado, una pregunta sobre una variable con dos estados posibles, o el final.*

3.1. VARIABLES Y ESTADOS. En cada momento el programa está en un solo estado. Los nombres de estados y variables los elige quien escribe el programa. El programa más corto escribe una letra y termina:

```
~~~> (DA1)           ;; empezar en el paso DA1
(DA1) !'F' x_x       ;; escribir F y terminar
```

3.2. LAS ÓRDENES. La tabla 1 reúne todo lo que se puede escribir. Un paso tiene siempre la misma forma: *su nombre entre paréntesis, lo que hace y cómo termina*. Por ejemplo, (VA3) +alfa ~> (VA2) se lee «en el paso VA3, suma 1 a alfa y ve al paso VA2».

TABLA 1. Las órdenes de DROSOPHILA-LANG y su equivalente en programación.

Escribes	Qué hace	En Python o Java
§ r = 0	crea la variable r a 0	r = 0
~~~> (X)	el programa empieza en el paso X	main
+r, -r	suma 1, resta 1 (sin bajar de 0)	r++, r--
<r	lee el siguiente número de la entrada y lo suma a r	r += input()
!"hola"	escribe un texto fijo	print("hola")
!*	mueve la trompa una vez; el intérprete cuenta cuántas	contador++
~> (Y)	va al paso Y	goto Y
?r ~> (A)   (B)	si $r > 0$ , le resta 1 y va a A; si $r = 0$ , va a B	if / else
x_x	termina el programa	exit()

La pregunta ? es la única forma de tomar una decisión, y solo sabe preguntar una cosa: «¿es mayor que cero?». Todo lo demás (comparar dos números, multiplicar...) se construye con bucles a partir de ella, y el Apartado 5 demuestra que eso basta. Los símbolos siguen la metáfora del olfato: la flecha ondulada ~> es algo que viaja, como un olor; la larga ~~~> es el *soplo* que pone el programa en marcha; los paréntesis rodean

el nombre de un glómérulo; y `x_x`, el emoticono de los ojos en cruz, es la muerte del programa.

Los datos de entrada forman una cola externa de longitud arbitraria, de la que cada `<` saca el siguiente; las variables son lo que el programa recuerda a la vez, y caben como mucho 16 (Apartado 5.3).

**3.3. UN EJEMPLO COMPLETO: LA SUMA.** El programa `suma.dros` lee dos números, los suma y escribe el resultado:

---

```
$ alfa = 0
$ beta = 0
~~~> (DM1)
(DM1) <alfa <beta ~> (VA2)
(VA2) ?beta ~> (VA3) | (DL5)
(VA3) +alfa ~> (VA2)
(DL5) !"suma: " ?alfa ~> (DA2) | (VM5d)
(DA1) ?alfa ~> (DA2) | (VM7d)
(DA2) !* ~> (DA1)
(VM5d) !'0' ~> (VM7d)
(VM7d) !'\n' x_x
```

---

Con la entrada (3, 4), la mosca guarda  $\alpha = 3$  y  $\beta = 4$ . El bucle  $VA2 \leftrightarrow VA3$  pasa las cuatro unidades de  $\beta$  a  $\alpha$ , una a una, hasta que  $\beta$  vale 0 y  $\alpha$  vale 7. No existe una orden «sumar»: sumar es pasar unidades de una variable a otra. El lenguaje tampoco puede escribir directamente el valor de una variable, así que el final vacía  $\alpha$  moviendo la trompa de la mosca una vez por unidad; el intérprete cuenta los movimientos y escribe `suma: 7`. La Figura 2 lo dibuja como un diagrama de estados.

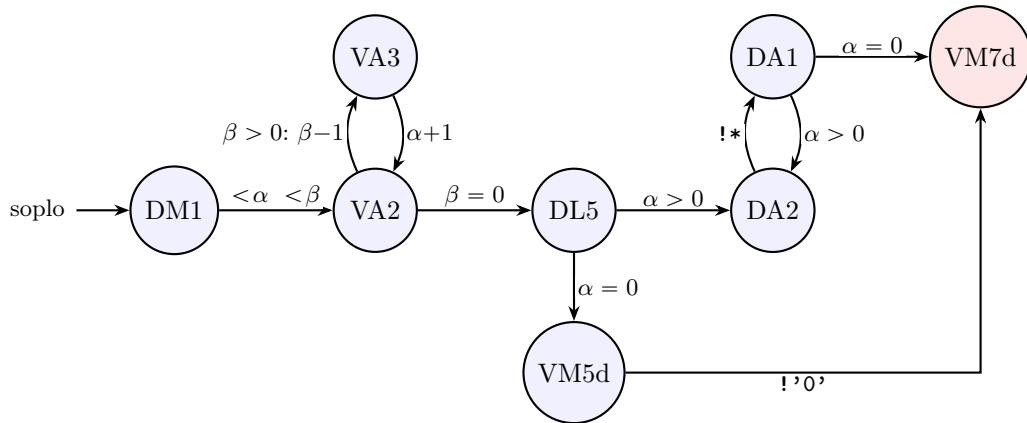


FIGURA 2. Diagrama de estados de `suma.dros`. Cada círculo es un glómérulo; las flechas indican a dónde pasa el control y qué condición o acción lo acompaña. En DL5 se escribe `suma:` antes de la primera pregunta. VM7d escribe el salto de línea y termina.

Los nombres de los pasos (DM1, VA2...) son glómérulos reales, pero no tienen una función fija: VA2 no «sirve para bucles». Lo que hace un paso lo decide lo que se escribe en su línea; el nombre solo decide qué neurona lo ejecuta. Con nombres inventados, como (BUCLE) o (SUMAR), el programa haría exactamente lo mismo. La gramática completa, en notación EBNF, está en el repositorio del código.

## 4. CÓMO SE EJECUTA

Un programa se ejecuta con una orden como `./mosca.sh suma.dros --olor "3 4"`. El lanzador `mosca.sh` solo activa Python; todo lo relacionado con neuronas está en el intérprete, `drosophila.py`. FlyWire no proporciona ningún programa, sino datos: cada neurona, su tipo, su posición y sus conexiones (unos 135 MB). El intérprete lee el programa, asigna a cada pieza una neurona real libre, añade entre ellas las conexiones del programa y simula el cerebro paso a paso hasta que dispara la Fibra Gigante.

Si dos neuronas elegidas ya están conectadas en la mosca, se aprovecha esa conexión; si no, el intérprete añade un cable directo, en una capa aparte que no modifica los datos de FlyWire. En `suma.dros`, de las 84 conexiones que necesita el programa, 8 existen en la mosca y 76 las añade el intérprete. Para elegir las neuronas, el intérprete busca dentro de cada tipo celular las que ya están conectadas entre sí y, si no hay ninguna, toma la primera libre en un orden fijo, así que el mismo programa elige siempre las mismas. Cada neurona tiene un solo papel, igual que un compilador asigna una posición de memoria distinta a cada variable.

**4.1. EL MODELO DE NEURONA.** El intérprete simula el cerebro en tiempo discreto,  $t = 0, 1, 2, \dots$ . En cada paso, cada neurona  $i$  está encendida o apagada,  $s_i(t) \in \{0, 1\}$ , y su estado en  $t + 1$  depende solo de las neuronas que dispararon en  $t$ . Hay dos clases de neuronas: las *reclutadas*, a las que el intérprete asigna un papel en el programa, y las *naturales*, que son todas las demás.

*Conexiones reales.* FlyWire da, para cada par  $j \rightarrow i$ , el número de sinapsis  $c_{ji}$  y el signo  $\varepsilon_j \in \{+1, -1\}$  de la neurona  $j$  (excitadora o inhibidora). El intérprete toma estos datos del repositorio que acompaña al modelo de Shiu *et al.* [3], conserva las conexiones con al menos cinco sinapsis y normaliza cada peso por la entrada total de la neurona que lo recibe,  $\tilde{w}_{ij} = \varepsilon_j c_{ji} / \sum_k c_{ki}$ . Así  $\sum_j |\tilde{w}_{ij}| = 1$ , y  $\sum_j \tilde{w}_{ij} s_j(t)$  es la fracción neta de las sinapsis de entrada de  $i$  que están activas.

*Neuronas naturales.* Cada una acumula esa entrada en un potencial con fuga (en cada paso conserva la mitad del anterior) y dispara cuando supera el umbral 0,08; después queda dos pasos en silencio (periodo refractario). Es una versión muy simplificada del modelo de integración y disparo de Shiu *et al.*: sirve para ver cómo reacciona el cerebro al programa, no para reproducir tasas de disparo reales.

*Neuronas reclutadas.* El programa se instala como un circuito extra con pesos enteros  $W_{ij}$  que fija el intérprete. Cada neurona reclutada calcula

$$h_i(t) = \underbrace{\sum_j W_{ij} s_j(t)}_{\text{programa}} + b_i + I_i(t) + \underbrace{\kappa \sum_j \tilde{w}_{ij} s_j(t)}_{\text{cerebro}}, \quad \kappa = \frac{1}{8}, \quad (4.1)$$

donde  $b_i \in \{-\frac{1}{2}, -\frac{3}{2}\}$  es un sesgo e  $I_i(t)$  un estímulo externo entero (el soplo inicial y la entrada olfativa), y dispara con probabilidad

$$\Pr[s_i(t+1) = 1] = \sigma(\beta h_i(t)) = \frac{1}{1 + e^{-\beta h_i(t)}}, \quad \beta = 64. \quad (4.2)$$

El parámetro  $\beta$  es la inversa de una temperatura que mide el ruido: cuando  $\beta \rightarrow \infty$  la neurona se vuelve una puerta de umbral exacta, que dispara si y solo si  $h_i > 0$ , como la neurona de McCulloch y Pitts [4]. El factor  $\kappa$  reduce la influencia del cerebro natural sobre el programa a un octavo; el Lema 5.2 muestra que, en el límite  $\beta \rightarrow \infty$ , eso basta

para que nunca cambie una decisión, y el Corolario 5.3 acota la probabilidad de que ocurra con  $\beta$  finito.

**4.2. LA MEMORIA.** Cada variable  $r$  es el peso  $w_r \in \mathbb{N}$  de una sinapsis real  $KC_r \rightarrow MBON_r$ , la misma que usa la mosca para recordar un olor.  $PAM_r$  suma uno y  $PPL1_r$  resta uno: si  $PAM_r$  o  $PPL1_r$  disparan en el paso  $t$ , el peso cambia según  $w_r(t+1) = \max(0, w_r(t) + s_{PAM_r}(t) - s_{PPL1_r}(t))$ , así que sube o baja de uno en uno y nunca es negativo. Para leerla, el programa enciende  $KC_r$ ; la neurona  $MBON_r$ , con sesgo  $-\frac{1}{2}$ , recibe  $w_r - \frac{1}{2}$  y dispara si y solo si  $w_r \geq 1$ . Una sola neurona responde así a la pregunta «¿es  $r \neq 0$ ?», pero no dice cuánto vale  $r$ : para saberlo hay que vaciar la variable contando, que es lo que hace el final de la suma. En la simulación de la suma, la variable alfa usa una célula de Kenyon de tipo KCg-s2, la MBON05, la PAM08 y la PPL108.

**4.3. DOS CAPAS DE ACTIVIDAD.** En cada paso se actualizan a la vez las 138 639 neuronas, y en el cerebro conviven dos capas de actividad:

- **El programa.** Una única señal recorre las neuronas de control por los cables del programa, modifica variables al pasar por las PAM y PPL1, y en cada pregunta consulta una MBON para elegir el siguiente paso. En la suma son 51 neuronas.
- **El resto del cerebro.** Cada neurona del programa excita o inhibe a sus vecinas reales, y se forman ondas de actividad que se apagan en pocos pasos. Con la suma de 3 y 4 llegan a encenderse unas 600 neuronas a la vez, y 3658 distintas en total.

La segunda capa no altera la primera: por (4.1), la parte del programa siempre está al menos a  $\frac{1}{2}$  del umbral, y la del cerebro nunca supera  $\frac{1}{8}$  (Lema 5.2). Por eso el resultado no depende de la actividad natural.

**4.4. CÓMO TERMINA.** La salida de números usa la motoneurona MN9, que en la mosca extiende la probóscide, la «trompa» con la que come [8]. Cada  $!*$  la dispara una vez y el intérprete cuenta los disparos, como quien cuenta con palitos. El final normal es la Fibra Gigante, la neurona que en la mosca desencadena el salto de huida [9]: cuando dispara, el intérprete para. También se detiene si el programa cae en silencio («coma»), si repite exactamente un estado anterior («ciclo», un bucle infinito) o si agota el número máximo de pasos.

## 5. ¿POR QUÉ ES TURING-COMPLETO?

Un lenguaje es *Turing-completo* si puede calcular cualquier cosa que calcule un ordenador, suponiendo que tiene tiempo y memoria de sobra. La forma habitual de demostrarlo no es construir un ordenador entero, sino mostrar que el lenguaje puede imitar otro sistema del que ya se sabe que es universal. Aquí ese sistema son las máquinas de Minsky, porque sus instrucciones coinciden casi una a una con las de DROSOPHILA-LANG.

**Definición 5.1 (Máquina de Minsky).** Una máquina de Minsky es un par  $M = (Q, \mu)$ , donde  $Q$  es un conjunto finito de estados con un estado inicial  $q_0$ , y  $\mu$  asigna a cada  $q \in Q$  una instrucción sobre registros  $r_1, \dots, r_m \in \mathbb{N}$ :

- $INC(r, q')$ : suma 1 al registro  $r$  y pasa al estado  $q'$ ;
- $JZDEC(r, q', q'')$ : si  $r > 0$ , le resta 1 y pasa a  $q'$ ; si  $r = 0$ , pasa a  $q''$ ;
- $HALT$ : se detiene.

Una configuración es un par  $(q, \mathbf{x})$  con  $q \in Q$  y  $\mathbf{x} \in \mathbb{N}^m$  el contenido de los registros; cada instrucción define la configuración siguiente.

**5.1. MÁQUINAS DE MINSKY.** Minsky demostró en 1967 que esto basta, incluso con solo dos registros, para imitar cualquier máquina de Turing y, por tanto, cualquier ordenador [10]. Korec construyó además en 1996 una máquina de Minsky *universal* concreta,  $U_{32}$ , de 32 instrucciones y 8 registros: una sola máquina fija que, si recibe en sus registros la descripción de otra máquina y sus datos, la ejecuta [11]. Es lo mismo que hace un ordenador con un programa cualquiera.

**5.2. LA TRADUCCIÓN.** Cada instrucción de Minsky tiene su equivalente directo en DROSOPHILA-LANG (tabla 2): los registros son variables, los estados son pasos, y la instrucción JZDEC es exactamente la pregunta  $?$ . El programa `suma.dros` es, de hecho, una máquina de Minsky con algo de entrada y salida alrededor: su núcleo son  $VA2 : JZDEC(\beta, VA3, DL5)$  y  $VA3 : INC(\alpha, VA2)$ .

TABLA 2. Cómo se traduce cada instrucción de Minsky a DROSOPHILA-LANG y qué neuronas la ejecutan.

Minsky	DROSOPHILA-LANG	Neuronas
$INC(r, q')$	$(q) \ +r \ \leadsto (q')$	el paso enciende la PAM de $r$
$JZDEC(r, q', q'')$	$(q) \ ?r \ \leadsto (q') \mid (q'')$	el circuito de la Figura 3
HALT	$(q) \ x_x$	la Fibra Gigante

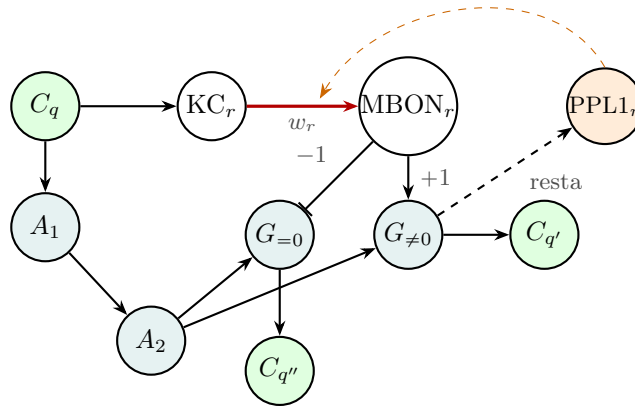


FIGURA 3. El circuito que ejecuta una pregunta  $?r \leadsto (A) \mid (B)$ . La neurona del paso ( $C_q$ ) enciende la KC, que lee la variable: la MBON solo responde si la variable no es cero. A la vez, la señal pasa por dos neuronas de retardo ( $A_1, A_2$ ) para llegar a las dos compuertas justo cuando llega la respuesta de la MBON. Si la MBON respondió, se enciende la compuerta de arriba, que resta 1 mediante la PPL1 y pasa el turno al paso A ( $C_{q'}$ ); si no, se enciende la de abajo y pasa el turno al paso B ( $C_{q''}$ ). Las compuertas y los retardos (en turquesa) son neuronas locales del cuerno lateral.

Formalmente, la traducción asigna a cada registro  $r$  su engrama ( $KC_r, MBON_r, PAM_r, PPL1_r$  y el peso  $w_r$  de la Apartado 4.2) y a cada estado  $q$  una neurona de control  $C_q$  con

sesgo  $-\frac{1}{2}$ .  $\text{INC}(r, q')$  se convierte en las conexiones  $C_q \rightarrow \text{PAM}_r$  y  $C_q \rightarrow C_{q'}$ ;  $\text{HALT}$ , en  $C_q \rightarrow \text{GF}$ , donde GF es la Fibra Gigante; y  $\text{JZDEC}(r, q', q'')$ , en el circuito de la Figura 3. En él, las dos compuertas son puertas de umbral: en el límite  $\beta \rightarrow \infty$ ,  $G_{\neq 0}$  dispara si y solo si  $A_2 + \text{MBON}_r - \frac{3}{2} > 0$  (una puerta Y) y  $G_{=0}$  si y solo si  $A_2 - \text{MBON}_r - \frac{1}{2} > 0$  (« $A_2$  y no  $\text{MBON}_r$ »).

Queda comprobar que el circuito se comporta como la máquina ideal. Podrían fallar tres cosas: que el ruido o el resto del cerebro inviertan una decisión, que queden dos estados activos a la vez, o que una pregunta lea una variable antes de que termine de escribirse. El primer punto es el único que necesita un cálculo.

**Lema 5.2 (Margen).** *Para toda neurona reclutada  $i$  y todo paso  $t$ ,  $|h_i(t)| \geq \frac{3}{8}$ .*

*Demostración.* En (4.1), los pesos  $W_{ij}$ , los estímulos  $I_i$  y los pesos de memoria  $w_r$  son enteros, y el sesgo  $b_i$  es un entero menos  $\frac{1}{2}$ . La parte del programa es, por tanto, de la forma  $n + \frac{1}{2}$  con  $n \in \mathbb{Z}$ , y su valor absoluto es al menos  $\frac{1}{2}$ . Como  $\sum_j |\tilde{w}_{ij}| = 1$  y  $s_j \in \{0, 1\}$ , la parte del cerebro cumple  $|\kappa \sum_j \tilde{w}_{ij} s_j| \leq \kappa = \frac{1}{8}$ . Luego  $|h_i| \geq \frac{1}{2} - \frac{1}{8} = \frac{3}{8}$ .  $\square$

**Corolario 5.3 (Probabilidad de error).** *Si un programa usa  $N$  neuronas reclutadas durante  $\tau$  pasos, la probabilidad de que alguna se comporte de forma distinta que en el límite  $\beta \rightarrow \infty$  es como mucho*

$$\tau N \sigma\left(-\frac{3\beta}{8}\right) \leq \tau N e^{-3\beta/8}. \quad (5.1)$$

*Demostración.* Por el Lema 5.2 y (4.2), cada neurona se equivoca en cada paso con probabilidad como mucho  $\sigma(-3\beta/8)$ ; la probabilidad de la unión de  $\tau N$  sucesos es como mucho la suma de sus probabilidades.  $\square$

Con  $\beta = 64$ , la cota por neurona y paso es  $\sigma(-24) \approx 3.8 \times 10^{-11}$ . Para `suma.dros` con entrada (3, 4), que usa  $N = 51$  neuronas durante  $\tau = 87$  pasos, (5.1) da una probabilidad total de error menor que  $2 \times 10^{-7}$ . La cota crece con  $\tau$ , así que una ejecución mucho más larga necesita un  $\beta$  mayor para el mismo margen. Los otros dos puntos se siguen de la construcción:

- **Un solo estado activo.** En INC y en los saltos, cada neurona de control conecta con una única neurona de control sucesora; en una pregunta, los dos sucesores cuelgan de las compuertas, y estas son excluyentes, porque una exige  $\text{MBON}_r = 1$  y la otra  $\text{MBON}_r = 0$ . Por inducción sobre  $t$ , en cada paso hay a lo sumo una  $C_q$  activa.
- **Escribir antes de leer.** El cambio de  $w_r$  se aplica en el paso siguiente al disparo de la  $\text{PAM}_r$  o  $\text{PPL1}_r$ , y los retardos  $A_1$ ,  $A_2$  hacen que la siguiente lectura de  $\text{KC}_r$  llegue después de ese cambio.

**5.3. EL CEREBRO ES FINITO.** Un ordenador teórico tiene memoria ilimitada, pero el cerebro de la mosca tiene un número finito de neuronas, y cada pieza del programa gasta alguna. Los dos recursos que antes se agotan son las neuronas PPL1, una por variable, de las que hay 16; y las neuronas locales del cuerno lateral, cuatro por pregunta, de las que hay 514. Por tanto, un programa puede tener como mucho 16 variables y 128 preguntas ( $514/4 = 128,5$ ). El resto de tipos de neurona sobra.

**Teorema 5.4.** DROSOPHILA-LANG es Turing-completo.

*Demostración.* Toda máquina de Minsky con como mucho 16 registros y 128 instrucciones JZDEC se puede traducir a DROSOPHILA-LANG con la tabla 2, y cabe en el cerebro. Por el Lema 5.2 y los dos puntos que lo siguen, en el límite  $\beta \rightarrow \infty$  el programa traducido pasa por los mismos estados que la máquina, con sus variables siempre iguales a los registros, y dispara la Fibra Gigante exactamente cuando la máquina se detiene. No hace falta que quepa cualquier máquina: basta con que quepa una universal. La  $U_{32}$  de Korec tiene 8 registros y 32 instrucciones, así que cabe; DROSOPHILA-LANG puede simularla, y con ella cualquier cálculo. La memoria ilimitada que exige la definición está en la fuerza de las conexiones, que puede crecer sin límite.  $\square$

Como todo lenguaje Turing-completo, DROSOPHILA-LANG hereda también el problema de la parada [12]: no existe un método general para saber de antemano si un programa terminará. El intérprete detecta los casos fáciles (silencio o un estado repetido), pero no todos.

## 6. RESULTADOS

**6.1. PROGRAMAS DE EJEMPLO.** La tabla 3 resume la ejecución de los programas de ejemplo. Todos producen la salida correcta. `longitud.dros` lee un texto, en el que cada byte llega como una concentración de olor, y cuenta sus bytes.

TABLA 3. Ejecución de los programas de ejemplo. *Pasos*: pasos hasta la muerte; *reclut.*: neuronas reclutadas; *despiertas*: neuronas naturales que dispararon al menos una vez; *reales*: conexiones del programa que ya existían en el conectoma.

Programa	Entrada	Salida	Pasos	Reclut.	Despiertas	Reales	Tiempo
<code>efe.dros</code>	—	F	3	13	76	1/6	<0.01 s
<code>hola.dros</code>	—	¡Hola, Mundo!	17	27	860	1/83	0.01 s
<code>suma.dros</code>	0, 0	suma: 0	26	51	3248	8/84	0.01 s
<code>suma.dros</code>	3, 4	suma: 7	87	51	3658	8/84	0.04 s
<code>suma.dros</code>	12, 30	suma: 42	427	51	3970	8/84	0.22 s
<code>suma.dros</code>	200, 300	suma: 500	4525	51	3971	8/84	2.26 s
<code>longitud.dros</code>	mosca	bytes: 5	2747	55	2173	10/98	1.24 s

**6.2. EL CEREBRO REACCIONA.** Aunque el cerebro natural no decide nada, responde al programa: en `suma.dros` con entrada (3, 4) llegan a disparar unas 600 neuronas naturales a la vez y 3658 distintas en total. Los picos coinciden con el inicio, con la lectura de cada número y con cada vuelta del bucle de transferencia, porque en esos momentos disparan neuronas de proyección, que en el conectoma conectan con cientos de células de Kenyon.

**6.3. COMPROBACIÓN DE LA DEMOSTRACIÓN.** El Apartado 5 demuestra que la traducción funciona para cualquier máquina de Minsky que quepa en el cerebro; este experimento lo comprueba sobre el mapa real. El guion `minsky.py` genera máquinas aleatorias, de 2 a 12 estados y de 1 a 3 registros, las traduce con la tabla 2, las ejecuta y compara los registros finales con una simulación directa en Python. De 3000 máquinas

generadas, 1333 se detienen en menos de 2000 pasos, y las 1333 dieron el mismo resultado en el cerebro que en la simulación directa.

El límite del Apartado 5.3 también se comprobó en la práctica: un programa con 128 preguntas `?r` se instala y termina, y uno con 129 falla porque el cerebro se queda sin neuronas del cuerno lateral.

**6.4. ROBUSTEZ FRENTE AL RUIDO.** Las neuronas reclutadas son estocásticas: por (4.2), disparan con una probabilidad que depende de  $\beta$ , y con  $\beta$  pequeño se equivocan a menudo. El valor por defecto es  $\beta = 64$ , para el que el Corolario 5.3 garantiza un error despreciable; este experimento mide qué pasa por debajo. Para cada valor de  $\beta$  se ejecutó `suma.dros` con entrada (3, 4) y 40 semillas de ruido distintas, y se contó cuántas ejecuciones escriben `suma: 7`. La transición es brusca: todas aciertan para  $\beta \geq 24$ , la mitad para  $\beta = 16$  y ninguna para  $\beta \leq 12$ . El fallo típico es que la Fibra Gigante dispare antes de tiempo y la mosca «huya» sin terminar la cuenta: así acaban 39 de los 40 fallos con  $\beta = 12$  y 18 de los 20 con  $\beta = 16$ . La cota (5.1) es coherente con esto: para  $\beta = 24$  vale  $87 \cdot 51 \cdot e^{-9} \approx 0,55$ , que ya no garantiza nada aunque en la práctica todas las ejecuciones acierten, mientras que para  $\beta = 64$  es menor que  $2 \times 10^{-7}$ .

## 7. DISCUSIÓN

**7.1. ¿CALCULA LA MOSCA?** Son reales las neuronas, con su identidad, su tipo y su posición, y las 2700513 conexiones por las que se propagan las ondas de actividad del Apartado 6. Pero el circuito que calcula es casi todo añadido: de las 84 conexiones que usa `suma.dros`, solo 8 existen en la mosca, y el resto del cerebro solo llega al programa atenuado por  $\kappa = \frac{1}{8}$ , que por el Lema 5.2 no cambia ninguna decisión. La mosca no suma: su cerebro aporta los componentes y el intérprete aporta el cableado. Tampoco se entrena a la mosca: en el animal vivo las PAM y las PPL1 responden a premios y castigos, mientras que aquí el intérprete las activa directamente. Se toma prestado el mecanismo con el que la mosca aprende, no su significado. Por la misma razón, el programa no podría ejecutarse en una mosca viva: la optogenética permite activar neuronas concretas, pero no crear los 76 cables que faltan.

**7.2. QUÉ HACE FALTA PARA CALCULAR.** Para calcular cualquier cosa basta un programa finito que sepa sumar uno, restar uno y preguntar si un número es cero, esté hecho de neuronas, de transistores o de piedras. Lo difícil no es la universalidad sino la eficiencia: como DROSOPHILA-LANG cuenta de uno en uno, sumar 200 y 300 le cuesta 4525 pasos. Y en DROSOPHILA-LANG los límites no los pone la memoria de un ordenador sino la anatomía: 16 variables porque hay 16 neuronas PPL1, y 128 preguntas porque hay 514 neuronas locales en el cuerno lateral.

**7.3. LIMITACIONES.** La plasticidad está simplificada: en la mosca, la dopamina debilita la conexión  $KC \rightarrow MBON$  de las células de Kenyon activas [7], mientras que aquí una familia suma y la otra resta. El modelo de neurona es mínimo y no está ajustado a datos fisiológicos, a diferencia del de Shiu *et al.* [3]. Y la salida solo puede ser texto fijo o un número, así que un programa no puede calcular una letra y escribirla.

## 8. CONCLUSIONES

*We can only see a short distance ahead, but we can see plenty there that needs to be done.*³ – A. M. Turing [13]

DROSOPHILA-LANG es un lenguaje esotérico cuyos programas se ejecutan sobre el conectoma completo de la mosca de la fruta. Su sintaxis traduce la anatomía: los estados son glomérulos, la memoria es una sinapsis que la dopamina modifica, las preguntas las responden las MBON y el cuerno lateral, y el programa termina con la huida de la mosca. Hemos demostrado que es Turing-completo simulando máquinas de Minsky, con un argumento que tiene en cuenta que el cerebro es finito, y lo hemos comprobado con 1333 máquinas aleatorias ejecutadas sobre el conectoma real.

Quedan abiertas varias extensiones: elegir las neuronas del programa para maximizar las conexiones que ya existen en el conectoma, dejar que la actividad natural participe en las decisiones en lugar de solo observarlas, y sustituir el modelo de neurona por uno ajustado a datos fisiológicos.

## REFERENCIAS

- [1] Dorkenwald, S. et al. Neuronal wiring diagram of an adult brain. *Nature* 634.8032 (2024), págs. 124-138. DOI: 10.1038/s41586-024-07558-y.
- [2] Schlegel, P. et al. Whole-brain annotation and multi-connectome cell typing of *Drosophila*. *Nature* 634.8032 (2024), págs. 139-152. DOI: 10.1038/s41586-024-07686-5.
- [3] Shiu, P. K. et al. A *Drosophila* computational brain model reveals sensorimotor processing. *Nature* 634.8032 (2024), págs. 210-219. DOI: 10.1038/s41586-024-07763-9.
- [4] McCulloch, W. S. y Pitts, W. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics* 5.4 (1943), págs. 115-133. DOI: 10.1007/bf02478259.
- [5] Kurtovic, A., Widmer, A. y Dickson, B. J. A single class of olfactory neurons mediates behavioural responses to a *Drosophila* sex pheromone. *Nature* 446.7135 (2007), págs. 542-546. DOI: 10.1038/nature05672.
- [6] Aso, Y. et al. The neuronal architecture of the mushroom body provides a logic for associative learning. *eLife* 3 (2014). DOI: 10.7554/elife.04577.
- [7] Hige, T. et al. Heterosynaptic Plasticity Underlies Aversive Olfactory Learning in *Drosophila*. *Neuron* 88.5 (2015), págs. 985-998. DOI: 10.1016/j.neuron.2015.11.003.
- [8] Gordon, M. D. y Scott, K. Motor Control in a *Drosophila* Taste Circuit. *Neuron* 61.3 (2009), págs. 373-384. DOI: 10.1016/j.neuron.2008.12.033.
- [9] Allen, M. J., Godenschwege, T. A., Tanouye, M. A. y Phelan, P. Making an escape: Development and function of the *Drosophila* giant fibre system. *Seminars in Cell & Developmental Biology* 17.1 (2006), págs. 31-41. DOI: 10.1016/j.semcdb.2005.11.011.
- [10] Minsky, M. L. *Computation: Finite and Infinite Machines*. Englewood Cliffs, NJ: Prentice-Hall, 1967.
- [11] Korec, I. Small universal register machines. *Theoretical Computer Science* 168.2 (1996), págs. 267-301. DOI: 10.1016/s0304-3975(96)00080-1.
- [12] Turing, A. M. On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society* s2-42.1 (1937), págs. 230-265. DOI: 10.1112/plms/s2-42.1.230.
- [13] Turing, A. M. Computing Machinery and Intelligence. *Mind* 59.236 (1950), págs. 433-460. DOI: 10.1093/mind/LIX.236.433.

---

³«Solo podemos ver un poco más allá, pero ya vemos mucho por hacer» (traducción propia).